

Testing Tree-Consistency with k Missing Quartets

Chuang-Chieh Lin

Department of Computer Science and Information Engineering,
National Chung Cheng University, Ming-Hsiung, Chiayi, Taiwan.
lincc@cs.ccu.edu.tw

Abstract

In the past decade, property testing has been a very active field in theoretical computer science. It considers the following task: given a function f over a domain D , a property $\mathcal{P}$ and a parameter $0 < \epsilon < 1$, by examining function values of f over $o(|D|)$ elements in D , determine whether f satisfies $\mathcal{P}$ or differs from any one which satisfies $\mathcal{P}$ in at least $\epsilon|D|$ elements. An algorithm that fulfills this task is called a property tester. We focus on tree-consistency of quartet topologies, which is a combinatorial property originating from evolutionary tree construction. The input function is f_{tc}^Q , which assigns one of the three possible topologies or a null symbol $\emptyset$ for every quartet over an n -taxon set S . A quartet $\mathbf{t}$ with $f_{tc}^Q(\mathbf{t}) = \emptyset$ is called a missing quartet. We say that f_{tc}^Q is tree-consistent if there exists an evolutionary tree T whose induced quartet topologies coincide with f_{tc}^Q . In this paper, we show that when the set of k missing quartets is given, there is an $O(k3^kn^3/\epsilon)$ property tester for tree-consistency.

1 Introduction

1.1 Property testing

Let $\mathcal{F}$ be the set of all functions with the same domain D . Let $\mathcal{P}$ be a fixed property of functions in $\mathcal{F}$, which can be viewed as a subset of $\mathcal{F}$. For two functions $f, g \in \mathcal{F}$, let $\delta(f, g)$ denote the fraction of elements in D for which f and g have different values. Obviously the range of δ is $[0, 1]$. Then for a function $f \in \mathcal{F}$, we define $\Delta(f, \mathcal{P}) = \min_{g \in \mathcal{P}} \delta(f, g)$. We say that f satisfies the property $\mathcal{P}$ if $\Delta(f, \mathcal{P}) = 0$. We say that f is ϵ -far from satisfying $\mathcal{P}$ if $\Delta(f, \mathcal{P}) \geq \epsilon$, otherwise f is said to be ϵ -close to satisfying $\mathcal{P}$. The complexity measure we concern is the *query complexity*, where

a query is an examination the function value of an element of D , and we assume that a query takes only constant time. The following definition of a *property tester* is given by Goldreich et al. [16].

Definition 1 ([16]). *Given a function $f \in \mathcal{F}$ and a parameter $0 < \epsilon < 1$, a property tester for $\mathcal{P}$ is an algorithm $\mathcal{M}$ making $o(|D|)$ queries¹, such that*

1. $\mathcal{M}$ returns “yes” with probability at least $2/3$ if $f \in \mathcal{P}$;
2. $\mathcal{M}$ returns “no” with probability at least $2/3$ if $\Delta(f, \mathcal{P}) \geq \epsilon$.

Moreover, we say that $\mathcal{M}$ is a one-sided-error property tester if it returns “yes” whenever f satisfies $\mathcal{P}$ with probability 1.

As clarified in [14, 20], the probability ‘ $2/3$ ’ of success can be substituted by any arbitrary constant strictly greater than $1/2$. In order to have success probability of $1 - \rho$ for any $\rho < 1/3$, we can simply execute $\mathcal{M}$ for $O(\log(1/\rho))$ times and return the majority of these outputs.

As mentioned in [20], property testing may be useful in some scenarios. For example, suppose we have a slow exact decision procedure and a property tester for a function. If the property tester answers “no”, then we know that with high probability the function does not have the property. In particular, for one-sided-error property testers, such a negative answer provides a witness that the function does not have the property, and therefore it is not necessary to run the slow decision procedure. Property testing is also useful when we can tolerate a small number of errors of the function values. In such a scenario, we only care whether the function is “good” (i.e., has the property) or “very bad” (i.e., ϵ -far from having the property).

¹Note that $o(\cdot)$ here is an asymptotic notation of “little-o”. For functions $f, g : \mathbb{N} \mapsto \mathbb{R}^+$, we denote by $f(x) = o(g(x))$ if $\lim_{x \rightarrow \infty} f(x)/g(x) = 0$.

The general notion of property testing was first formulated by Rubinfeld and Sudan [21], who were motivated by the connection to program checking [6]. The study on testing combinatorial objects was first introduced by Goldreich, Goldwasser, and Ron [16]. Property testing is a very active field in theoretical computer science, especially in testing graph properties. See [1, 14, 15, 20] for surveys.

1.2 Evolutionary trees and quartets

Determining the evolutionary relationship of a set of taxa is a very essential topic in computational biology. In order to model such relationships, *evolutionary trees* are widely considered. Roughly speaking, an evolutionary tree represents the course of evolution for a set of taxa over time. In an evolutionary tree, the leaves represent the taxa and the internal nodes represent the ancestors. Building an evolutionary tree for all the taxa has been regarded as a crucial and fundamental problem in computational biology.

We assume that every internal node of an evolutionary tree is of degree 3 due to the reason that events of taxon divergence are usually rare in practical cases, and multifurcations can be viewed as aggregates of bifurcations in some circumstances [23]. Rooted and unrooted evolutionary trees are both studied, however, construction of unrooted evolutionary trees is mostly considered. One crucial reason is that for the same set of leaves, there are fewer unrooted evolutionary trees than rooted ones (See [13] for further discussions). In this paper, we focus on unrooted evolutionary trees.

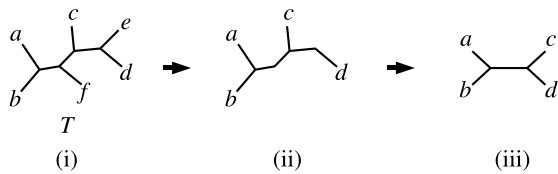


Fig. 1: (i) An evolutionary tree T ; (ii) The path structure connecting a, b, c, d in T ; (iii) The quartet topology of $\{a, b, c, d\}$ induced by T .

In practical cases, one can only analyze the evolutionary relation of a small set of taxa at one time. Hence, in order to observe the whole evolution course of all the taxa, one has to construct an evolutionary tree from a set of small subtrees. However, in reality some of the given small trees

may be erroneous, so an evolutionary tree consistent with all of them may not exist. Moreover, determining if such an evolutionary tree exists is proved to be **NP**-complete [22]. Therefore, one turns to seek for an evolutionary tree consistent with as many of the input small trees as possible. For practical and theoretical advantages, *quartet methods*, where the input consists of a set of unrooted four-leaf trees, are extensively used for this kind of optimization problem in the past decade [3–5, 7–12, 17–19, 22].

Let S be a set of n taxa. An evolutionary tree T over S is an *unrooted, leaf-labeled* tree such that the leaves of T are bijectively labeled by the taxa in S , and each internal node of T has degree three. A *quartet* is a set of four taxa in S . The *quartet topology* for $\{a, b, c, d\}$ induced by T is the path structure connecting a, b, c , and d in T (see Fig. 1 for an illustration). Equivalently, $\{a, b, c, d\}$ has the quartet topology $[ab|cd]$ with respect to T if and only if the path on T from a to b does not share any vertex with that from c to d . In this paper, a quartet $\{a, b, c, d\}$ is restricted to have one of the three topologies $[ab|cd]$, $[ac|bd]$, and $[ad|bc]$ (see Fig. 2), which are the possible bipartitions of $\{a, b, c, d\}$.

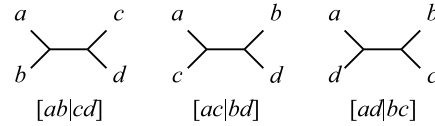


Fig. 2: Three topologies for the quartet $\{a, b, c, d\}$.

We denote by Q_T the set of quartet topologies induced by an evolutionary tree T . A set of quartet topologies Q is said to be *complete* (with respect to S) if Q contains exactly one topology for every quartet in S . We say that Q is *tree-consistent* [2] if there exists an evolutionary tree T such that $Q \subseteq Q_T$. Furthermore, we say that Q is *tree-like* [2] if $Q = Q_T$ for some evolutionary tree T . For example, if $S = \{a, b, c, d, e, f\}$ and $Q = \{[ab|cd], [ab|ce], [ab|cf], [ab|de], [ab|df], [ab|ef], [ac|de], [af|cd], [af|ce], [af|de], [bc|de], [bf|cd], [bf|ce], [bf|de], [cf|de]\}$, then Q is tree-like since it is exactly the set of quartet topologies induced by T in Fig. 1 (i). Let $\mathcal{P}_{tree}$ denote the set of all tree-like sets of quartet topologies over S . We call $\min_{Q^* \in \mathcal{P}_{tree}} |Q \setminus Q^*|$ the *error number* of Q . We call the quartet topologies in $Q \setminus Q^*$ the *quartet errors* of Q if $|Q \setminus Q^*|$ equals to the error number

of Q for $Q^* \in \mathcal{P}_{tree}$.

The setting of property testing of tree-consistency. For the three possible topologies of a quartet $\{a, b, c, d\}$, we denote the quartet topologies $[ab|cd]$, $[ac|bd]$, and $[ad|bc]$ by *type* 0, 1, and 2, respectively, according to their lexicographical order. We regard Q as a function $f_{tc}^Q : \{\{a, b, c, d\} \mid a, b, c, d \in S\} \rightarrow \{0, 1, 2, \emptyset\}$, where $f_{tc}^Q(\{a, b, c, d\}) \neq \emptyset$ is equal to the type of the topology of $\{a, b, c, d\}$ in Q , and $f_{tc}^Q(\{a, b, c, d\}) = \emptyset$ denotes that the quartet $\{a, b, c, d\}$ does not have a topology in Q . We call a quartet $\{a, b, c, d\}$ a *missing quartet* if $f_{tc}^Q(\{a, b, c, d\}) = \emptyset$. The domain size of the function f_{tc}^Q is then equal to $\binom{n}{4}$. By the above settings, a query of f_{tc}^Q here retrieves the topology of a quartet in Q . We utilize an array of $\binom{n}{4}$ entries, where each entry stores the type of the topology of a quartet over S or the null symbol $\emptyset$. We define that $\Delta(Q, \mathcal{P}_{tree}) = \min_{Q^* \in \mathcal{P}_{tree}} |Q \setminus Q^*| / \binom{n}{4}$. Clearly, Q is tree-consistent if and only if $\Delta(Q, \mathcal{P}_{tree}) = 0$. We say that Q is ϵ -far from being tree-consistent if the error number of Q is at least $\epsilon \binom{n}{4}$, that is, $\Delta(Q, \mathcal{P}_{tree}) \geq \epsilon$.

1.3 Related work

Given a set of quartet topologies Q , the *Quartet Compatibility Problem* (QCP) is a decision problem of determining whether Q is tree-consistent. The QCP problem is **NP**-complete [22], however, it is polynomially solvable when Q is complete [4, 12], in which case it amounts to decide if Q is tree-like. There are theoretical interests in dealing with the case where Q is complete. Any evolutionary tree T can be uniquely defined by the set of quartet topologies Q_T it induces [11], and some optimization problems, introduced in the following, are still **NP**-hard even when the input Q is complete. The *Maximum Quartet Consistency Problem* (MQC) is to construct an evolutionary tree T over S inducing as many quartet topologies in Q as possible. The *Minimum Quartet Inconsistency Problem* (MQI), which is a dual problem to the MQC problem, is to construct an evolutionary tree T over S such that the size of $Q \setminus Q_T$ is minimized. The MQC and MQI problems are **NP**-hard [5]. However, the MQC problem possesses a polynomial time approximation scheme (PTAS) [19], while the best approximation ratio for the MQI problem found so far is $O(n^2)$ [18].

Provided with a positive integer k as an additional part of the input, the *parameterized MQI*

problem is to determine whether the error number of Q is *at most* k . Gramm and Niedermeier [17] gave an $O(4^k n + n^4)$ fixed-parameter algorithm for this problem. Very recently, Chang et al. [8] improved the parameterized complexity of this problem by giving three more efficient fixed-parameter algorithms, which have time complexities $O(3.0446^k n + n^4)$, $O(2.0162^k n^3 + n^5)$, and $O^*((1 + \epsilon)^k)$ respectively, where $\epsilon > 0$ is an arbitrarily small constant².

Very recently, Chang et al. [9] devise the first one-sided-error property tester for testing if a complete set of quartet topologies is tree-like. Their tester runs in $O(n^3/\epsilon)$ time. It is still open that whether there exists a property tester for tree-likeness with query complexity independent of n .

1.4 Our contribution

In [9], Chang et al. only deals with the case that the input set of quartet topologies Q is complete. The difficulty of extending their result to incomplete Q 's is also clarified. In this paper, we introduce a variant of the original problem of testing tree-likeness of Q , which is called *testing tree-consistency with k missing quartets*. We show that, when there are k missing quartets with respect to Q which are given in advance, there is an $O(k3^k n^3/\epsilon)$ property tester for testing if Q is tree-consistent.

The rest of this paper is organized as follows. Preliminaries are given in Sect. 2. In Sect. 3, we present our property tester for testing tree-consistency with k missing quartets given in advance. Finally, concluding remarks are given in Sect. 4.

2 Preliminaries

A *quintet* is a set of five taxa in S . Given a complete set of quartet topologies Q over the taxon set S , we say that a quintet $\{a, b, c, d, e\} \subseteq S$ is *resolved* with respect to Q if the set of quartet topologies over $\{a, b, c, d, e\}$ in Q is tree-like. Otherwise, we say that $\{a, b, c, d, e\}$ is *unresolved* with respect to Q . Similar to the quartet topology, the *quintet topology* of a quintet $\{a, b, c, d, e\}$ induced by an evolutionary tree T is the path structure connecting a, b, c, d , and e in T . Without loss of generality, assume that we have $[ab|cd]$ induced by T , then there are five quintet topologies for the

²For two functions $f, g : (\mathbb{N}, \mathbb{N}) \rightarrow \mathbb{R}$, we write $f = O^*(g)$ if $f(n, k) = O(\text{poly}(n, k) \cdot g(n, k))$.

quintet $\{a, b, c, d, e\}$ induced by T since there are five positions for inserting e into the tree structure of $[ab|cd]$. Since there are three different topologies for the quartet $\{a, b, c, d\}$, there are fifteen quintet topologies for a quintet $\{a, b, c, d, e\}$ (see Fig. 3).

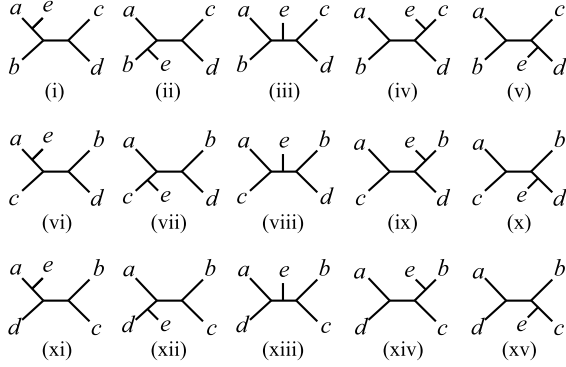


Fig. 3: The fifteen topologies for a quintet $\{a, b, c, d, e\}$.

Theorem 1 ([2]). *Given a set of taxa S , a complete set of quartet topologies Q over S , and a fixed taxon $\ell \in S$, Q is tree-like if and only if every quintet containing ℓ is resolved.*

Remarks. It follows from Theorem 1 that we can determine whether Q is tree-like by examining quintets with respect to Q . If Q is not tree-like (i.e., the error number of Q is at least one), by Theorem 1, we know that for any fixed taxon $\ell \in S$, there exists an unresolved quintet containing ℓ . Since there are $\binom{n-1}{4} = O(n^4)$ quintets containing a fixed taxon ℓ , it takes $O(n^4)$ time to deterministically check if Q is tree-like.

We denote by $T_{\text{miss}} = \{t_1, t_2, \dots, t_k\}$ the set of missing quartets. Our goal is to design a property tester to test whether a set of quartet topologies Q with k missing quartets given in advance is tree-consistent or not.

3 An $O(k3^k n^3/\epsilon)$ Property Tester for Instances with k Missing Quartets

3.1 The property tester

Algorithm *tree-like-tester* is the same as the tester for testing tree-likeness of a complete Q ,

except the number of iterations of the loop (in Line 2–7) increases to $O(kn^3/\epsilon)$. Its correctness is based on Theorem 1. One can obtain the following corollary easily from Theorem 4 in [9].

Algorithm 1 tree-like-tester [9]

Input: a complete set of quartet topologies Q .

- 1: pick an arbitrary taxon $\ell \in S$;
 - 2: **repeat**
 - 3: pick four taxa $s_1, s_2, s_3, s_4 \in S \setminus \{\ell\}$ uniformly at random;
 - 4: **if** the quintet $\{s_1, s_2, s_3, s_4, \ell\}$ is not resolved **then**
 - 5: return “no”;
 - 6: **end if**
 - 7: **until** the loop iterates for $\frac{72(k+1)}{\epsilon} n^3$ times
 - 8: return “yes”.
-

Corollary 1. *Algorithm tree-like-tester is a one-sided-error property tester for tree-likeness of quartet topologies, which makes at most $O(kn^3/\epsilon)$ queries. Furthermore, the tester returns “yes” with probability less than $1/3^{k+1}$ whenever the input Q is ϵ -far from being tree-like.*

There are 3^k possible assignments of topologies of the k missing quartets in T_{miss} . For $1 \leq i \leq 3^k$, we denote by $Q_{\text{miss}}(i) = \{q_1(i), q_2(i), \dots, q_k(i)\}$ the i th assignment of topologies of the missing quartets in T_{miss} , where $q_j(i)$ is the assigned topology of the quartet t_j in the i th assignment. We propose a property tester, denoted by *dense-consistency-tester*, for testing if Q is tree-consistent as follows.

Algorithm 2 dense-consistency-tester

Input: a complete set of quartet topologies Q , and a set of missing quartets $T_{\text{miss}} = \{t_1, t_2, \dots, t_k\}$.

- 1: **for** $i \leftarrow 1$ to 3^k **do**
 - 2: **if** *tree-like-tester*($Q \cup Q_{\text{miss}}(i)$) returns “yes” **then**
 - 3: return “yes”;
 - 4: **end if**
 - 5: **end for**
 - 6: return “no”.
-

3.2 Analysis of the algorithm

Theorem 2. *Given a set of quartet topologies Q and a set of k missing quartets $T_{\text{miss}} = \{t_1, t_2, \dots, t_k\}$, Algorithm dense-consistency-tester is an $O(k3^k n^3/\epsilon)$ property tester for testing if Q is tree-consistent.*

Proof. Since the loop in Line 1–5 of Algorithm `dense-consistency-tester` iterates 3^k times, by Corollary 1 we derive that the time complexity of the algorithm is $O(k3^kn^3/\epsilon)$. The correctness of the algorithm is proved as follows. First, consider the case that Q is tree-consistent. By definition, there exists an evolutionary tree T such that $Q \subset Q_T$. Since Algorithm `dense-consistency-tester` exhaustively tries every assignment of topologies for the missing quartets in T_{miss} , there must be some $j \in \{1, 2, \dots, 3^k\}$ such that $Q_{miss}(j) = Q_T \setminus Q$, which means that the algorithm must return “yes”. Consider the case that Q is ϵ -far from being tree-consistent. For each $j \in \{1, 2, \dots, 3^k\}$, $Q \cup Q_{miss}(j)$ must have at least $\epsilon \binom{n}{4}$ quartet errors. By the definition of a property tester and Corollary 1, Algorithm `tree-like-tester` is guaranteed to return “yes” with probability less than $(1/3)^{k+1}$ in this case. Thus, by the union bound, we obtain that Algorithm `dense-consistency-tester` returns “yes” with probability less than $(1/3)^{k+1} \cdot 3^k = 1/3$. Therefore, the theorem is proved. $\square$

4 Concluding Remarks

Be noted that to determine if Q is tree-consistent (i.e., the QCP problem) is **NP**-complete [22] if Q is not necessarily complete, however, the following arguments implies that it is polynomially solvable when the k missing quartets are given in advance. We substitute the condition of the if-statement at Line 2 of Algorithm `dense-consistency-tester` by using a deterministic $O(n^4)$ algorithm to check if $Q \cup Q_{miss}(i)$ is tree-like [8, 12]. Clearly, this problem can be solved in $O(3^kn^4)$ time, which is polynomial when k is a fixed constant.

Under the property testing model, the best time bound for testing if a complete set of quartet topologies is tree-like is $O(n^3/\epsilon)$ [9], which still depends on n , that is, the time complexity still depends on the size of the input. With k missing quartets that are given in advance, we show that there is an $O(k3^kn^3/\epsilon)$ one-sided-error property tester for testing if the input set of quartet topologies is tree-consistent, which slightly extends the results in [9]. Moreover, our property tester combines the notions of parameterized complexity and property testing. It will be interesting to show that either the dependency on n is inevitable for the complexity of testing such a property, or it is possible to obtain much more efficient property

testers by introducing suitable parameters.

Acknowledgments

We thank the anonymous referees for their reviews and comments, and thank Professor Maw-Shang Chang for the discussion about this problem.

References

- [1] N. Alon and A. Shapira: Homomorphisms in graph property testing - A survey. *Electronic Colloquium on Computational Complexity* (ECCC). Report No. 85 (2005).
- [2] H. J. Bandelt and A. Dress: Reconstructing the shape of a tree from observed dissimilarity data. *Adv. Appl. Math.* **7** (1986) 309–343.
- [3] A. Ben-Dor, B. Chor, D. Graur, R. Ophir, and D. Pelleg: From four-taxon trees to phylogenies: the case of mammalian evolution. In *Proceedings of the RECOMB*, 1998, 9–19. Also see: constructing phylogenies from quartets: elucidation of eutherian superordinal relationships. *J. Comput. Biol.* **5** (1998) 377–390.
- [4] V. Berry and O. Gascuel: Inferring evolutionary trees with strong combinatorial evidence. *Theoret. Comput. Sci.* **240** (2000) 271–298.
- [5] V. Berry, T. Jiang, P. E. Kearney, M. Li, and H. T. Wareham: Quartet cleaning: improved algorithms and simulations. In *Proceedings of the 7th Annual European Symposium on Algorithms* (ESA 1999), Lecture Notes in Comput. Sci., Vol. 1643, Springer Berlin, 1999, 313–324.
- [6] M. Blum, M. Luby, and R. Rubinfeld: Self-testing/correcting with applications to numerical problems. *J. Comput. System Sci.* **47** (1993) 549–595.
- [7] D. Bryant and M. Steel: Constructing optimal trees from quartets. *J. Algorithms* **38** (2001) 237–259.
- [8] M.-S. Chang, C.-C. Lin, and P. Rossmanith: New fixed-parameter algorithms for the minimum quartet inconsistency problem. *Theory Comput. Syst.* **47** (2010) 342–368.

- [9] M.-S. Chang, C.-C. Lin, and P. Rossmanith: A property tester for tree-likeness of quartet topologies. *Theory Comput. Syst.* To appear.
- [10] B. Cho: From quartets to phylogenetic trees. In *Proceedings of the 25th Conference on Current Trends in Theory and Practice of Informatics* (SOFSEM 1998), Lecture Notes in Comput. Sci., Vol. 1521, Springer, Berlin, 1998, 36–53.
- [11] H. Colonius and H. H. Schulze: Tree structures for proximity data. *British J. Math. Statist. Psycho.* **34** (1981) 167–180.
- [12] P. Erdős, M. Steel, L. Székely, and T. Warnow: A few logs suffice to build (almost) all trees (Part 1). *Random Structures Algorithms* **14** (1999) 153–184.
- [13] J. Felsenstein: *Inferring phylogenies*. Sinauer Associates, Inc. Publishers. Sunderland, Massachusetts.
- [14] E. Fischer: The art of uninformed decisions: A primer to property testing. *Bull. Eur. Assoc. Theoret. Comput. Sci.* (EATCS) **75** (2001) 97–126.
- [15] O. Goldreich: “Combinatorial property testing—a survey,” Randomization methods in algorithm design (Eds., P. Pardalos, S. Rajasekaran, and J. Rolin), DIMACS Series in Discrete Mathematics and Theoretical Computer Science, AMS, Providence, Vol. 43, 45–59. 1998.
- [16] O. Goldreich, S. Goldwasser, and D. Ron: Property testing and its connection to learning and approximation. *J. ACM* **45** (1998) 653–750.
- [17] J. Gramm and R. Niedermeier: A fixed-parameter algorithm for minimum quartet inconsistency. *J. Comput. System Sci.* **67** (2003) 723–741.
- [18] T. Jiang, P. E. Kearney, and M. Li: Some open problems in computational molecular biology. *J. Algorithms* **34** (2000) 194–201.
- [19] T. Jiang, P. E. Kearney, and M. Li: A polynomial time approximation scheme for inferring evolutionary tree from quartet topologies and its application. *SIAM J. Comput.* **30** (2001) 1942–1961.
- [20] D. Ron: Property testing, in *Handbook of Randomized Computing*, Vol. II, S. Rajasekaran, P. M. Pardalos, J. H. Reif, and J. D. P. Rolim, eds., Kluwer Academic Publishers, Dordrecht, The Netherlands, 2001, pp. 597–649.
- [21] R. Rubinfeld and M. Sudan: Robust characterization of polynomials with applications to program testing. *SIAM J. Comput.* **25** (1996) 252–271.
- [22] M. Steel: The complexity of reconstructing trees from qualitative characters and subtrees. *J. Classification* **9** (1992) 91–116.
- [23] R. W. H. Verwer and J. V. Pelt: Analysis of binary trees when occasional multifurcations can be considered as aggregates of bifurcations. *Bull. Math. Biol.* **52** (1990) 629–641.